

A Custom Kernel Support Vector Classifier with Optimized Hyperparameters for Binary Classification

Eduardo Henrique Basilio de Carvalho
Departamento de Engenharia Eletrônica
Universidade Federal de Minas Gerais
Belo Horizonte, Brasil
eduardohbc@ufmg.br

Abstract—This paper introduces a custom kernel support vector classifier designed for binary classification tasks. The proposed classifier utilizes a parameterized kernel where a critical hyperparameter is optimized to enhance class separability. Optimization of this hyperparameter is guided by one of two novel objective metrics: a 'spatial' metric based on inter-group and intra-group point distances in a projected space, or an 'axis' metric founded on the evenness of data distribution along projected axes. Initial kernel parameters are estimated using either covariance-based or scale-based methods. The custom classifier is integrated into a pipeline that includes a sequential preprocessing transformer for feature selection and scaling. An empirical evaluation is conducted using a standardized comparison framework, assessing performance against a standard support vector machine across multiple datasets, considering both `float32` and `float64` precision. The results indicate that while the custom kernel approaches can achieve performance statistically equivalent to the standard support vector machine in many cases, they do not consistently offer an accuracy advantage and typically incur higher computational costs. Numerical precision was observed to influence the stability and outcomes of the optimization process.

Index Terms—Support Vector Machines, Custom Kernel, Hyperparameter Optimization, Binary Classification, Performance Evaluation, Preprocessing.

I. INTRODUCTION

Support Vector Machines (SVMs) are powerful supervised learning models widely utilized for classification tasks, recognized for their strong theoretical underpinnings and robust empirical performance [1]. The efficacy of SVMs, especially non-linear variants, is heavily dependent on the selection of the kernel function and its associated parameters. The kernel function serves to implicitly map input data into a higher-dimensional feature space, wherein linear separation may become feasible. The Radial Basis Function (RBF) kernel represents a prevalent choice, with its performance being notably sensitive to the width parameter, frequently denoted as γ or related to a bandwidth h [1].

This paper introduces a novel custom kernel support vector classifier. In contrast to reliance on standard kernel functions with parameters chosen heuristically, this custom classifier employs a parameterized kernel whose hyperparameter, h , undergoes explicit optimization. This optimization process is

directed by custom objective metrics, formulated to quantify data separability or specific geometric properties within a kernel-induced space. Two such metrics are investigated: a 'spatial' objective and an 'axis' objective.

The custom kernel classifier is incorporated within a comprehensive machine learning pipeline, which features a multi-stage preprocessing transformer. This transformer is responsible for the removal of low-variance and highly-correlated features, data scaling, and dimensionality reduction through Principal Component Analysis (PCA). A systematic experimental framework is employed for the evaluation of the custom classifier configurations (utilizing 'spatial' and 'axis' objectives) in comparison to a standard SVM, with all models employing identical preprocessing. The comparison centers on classification accuracy and computation time across a diverse set of publicly available datasets (primarily from the UCI Machine Learning Repository [2], with specific datasets like QSAR Biodegradation [3] and KC1 from NASA MDP also included), and further explores the impact of numerical precision (`float32` versus `float64`) on the observed results.

The subsequent sections of this paper are structured as follows: Section II provides a detailed account of the methodology, covering the preprocessing stages, the formulation of the custom kernel, the objective metrics for hyperparameter optimization, the algorithm for the custom kernel support vector classifier, and the framework for performance evaluation. Section III presents and offers a discussion of the experimental results. Finally, Section IV provides a conclusion to the paper and suggests potential directions for future research.

II. METHODOLOGY

The proposed methodology comprises data preprocessing, a custom kernel formulation featuring hyperparameter optimization, the custom kernel support vector classifier, and a framework designed for performance evaluation.

A. Sequential Preprocessing Transformer

Data preprocessing is executed via a sequential transformer that applies four distinct stages:

- 1) **Low-Variance Feature Removal:** Features exhibiting a sample variance below a predefined threshold ($\tau_{var} = 0.05$) are eliminated.
- 2) **High-Correlation Feature Filtering:** Features are removed if the magnitude of their Pearson correlation with a preceding feature surpasses a specified limit ($\tau_{corr} = 0.95$).
- 3) **Data Scaling (Standardization):** The remaining features undergo standardization to achieve zero mean and unit standard deviation. In instances where the standard deviation of a feature is zero, its scaled values are set to zero.
- 4) **Principal Component Analysis (PCA):** PCA is employed for dimensionality reduction through the projection of data onto the principal components that correspond to the largest eigenvalues of the covariance matrix derived from the scaled data.

B. Kernel Formulation

The formulation of the custom kernel consists of an initial parameter estimation procedure and a parameterized kernel computation.

1) *Initial Kernel Parameter Estimation:* This procedure yields an estimation of a normalization factor η and an inverse matrix Ψ^{-1} from an input data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$. Support for two estimation types is provided:

- **Covariance-based Estimation:** Σ represents the empirical covariance matrix of $\mathbf{X}$. The normalization factor is $\eta = \frac{1}{(2\pi)^{D/2} \sqrt{\det(\Sigma)}}$, and the inverse matrix is $\Psi^{-1} = \Sigma^{-1}$. This method mandates that $\det(\Sigma)$ is both non-negligible and positive.
- **Scale-based Estimation:** The inverse matrix is defined as the identity matrix, $\Psi^{-1} = \mathbf{I}$. The normalization factor is $\eta = \frac{1}{(2\pi)^{D/2}}$.

2) *Parameterized Kernel Function:* Given two data matrices $\mathbf{X} \in \mathbb{R}^{N_X \times D}$ and $\mathbf{Y} \in \mathbb{R}^{N_Y \times D}$, initial parameters η_0, Ψ_0^{-1} , and a non-zero bandwidth hyperparameter h , this function computes a normalized kernel matrix $\mathbf{K} \in \mathbb{R}^{N_X \times N_Y}$.

- 1) **Parameter Scaling:** The initial parameters are scaled by h , yielding $\Psi_h^{-1} = \frac{\Psi_0^{-1}}{h^2}$ and $\eta_h = \frac{\eta_0}{h^D}$ (subject to boundary conditions for small h^D or $\eta_0 = 0$).
- 2) **Quadratic Form Matrix Computation (Q):** Elements $Q_{ij} = (\mathbf{x}_i - \mathbf{y}_j)^T \Psi_h^{-1} (\mathbf{x}_i - \mathbf{y}_j)$ are calculated.
- 3) **Unnormalized Kernel-like Value Calculation (P):** $P_{ij} = \eta_h \exp(-\frac{1}{2} Q_{ij})$.
- 4) **Final Normalization:** $K_{ij} = P_{ij} / P_{max}$, where $P_{max} = \max_{i,j} (P_{ij})$. If $P_{max} \approx 0$, then $K_{ij} = 0$.

C. Hyperparameter Optimization Objective Metrics

The optimization of the kernel hyperparameter h is guided by one of two metrics derived from data projections.

1) *Axis-based Objective Metric:* This metric corresponds to the "Comparative Spread Objective" for labeled feature sets, utilizing a "Vector Evenness Score" S_V . For a numerical sequence $V = (v_1, \dots, v_n)$:

- 1) Special cases: If $n = 0$, S_V is undefined. If $n = 1$, $S_V = 1.0$. Inputs containing NaN result in undefined scores.
- 2) For $n \geq 2$: V is sorted into V'_{sorted} . A difference sequence $\Delta = (\delta_j)$ is computed, where $\delta_j = v'_{(j+1)} - v'_{(j)}$.
- 3) The mean μ_Δ and population standard deviation σ_Δ of Δ are calculated (if $n = 2$, $\sigma_\Delta = 0$). The maximum difference δ_{max} is found.
- 4) Score components are: a Standard Deviation Score $S_\sigma = \frac{1}{1+\sigma_\Delta}$ (undefined if σ_Δ is NaN or Inf), and a Mean Spacing Score $S_\mu = (\mu_\Delta / \delta_{max})$ if $\delta_{max} \neq 0$, else 0.
- 5) The final Vector Evenness Score is $S_V = S_\sigma \times S_\mu$.

For the axis-based objective: Given a primary sequence X , a secondary sequence W , and binary labels L . Sub-sequences $V_{X_0} = \{x_i \mid l_i = 0\}$ and $V_{W_1} = \{w_i \mid l_i = 1\}$ are extracted. Evenness scores $S_{X_0} = S_V(V_{X_0})$ and $S_{W_1} = S_V(V_{W_1})$ are calculated. The objective value is $J_O = \frac{S_{X_0} + S_{W_1}}{2} - |S_{X_0} - S_{W_1}|$.

2) *Spatial Objective Metric:* This metric is the "Composite Separability and Cohesion Measure" J . Inputs consist of points P_{total} and labels L , which form groups G_0 and G_1 .

- 1) A Mean Inter-Group Distance (M_{inter}) is the average Euclidean distance μ_{G_0, G_1} between points in G_0 and G_1 . This requires $N_0, N_1 > 0$.
- 2) A Mean Intra-Group Distance for Group 0 ($M_{intra,0}$) is the average Euclidean distance μ_{G_0} among points in G_0 . This requires $N_0 \geq 2$.
- 3) A Mean Intra-Group Distance for Group 1 ($M_{intra,1}$) is the average Euclidean distance μ_{G_1} among points in G_1 . This requires $N_1 \geq 2$.
- 4) The objective value is $J = M_{inter} + \min(M_{intra,0}, M_{intra,1})$.

D. Custom Kernel Support Vector Classifier

The custom kernel support vector classifier integrates the custom kernel with an SVM.

- 1) **Initialization:** Parameters include bounds for h optimization (h_{min}, h_{max}), the type of initial kernel parameter estimation ('cov' or 'scale'), the objective metric for optimization ('spatial' or 'axis'), and arguments for the underlying standard SVM.
- 2) **Hyperparameter Optimization Objective Criterion:** This internal function evaluates a candidate hyperparameter $h_{candidate}$. Given $h_{candidate}$, training data $\mathbf{X}$, labels $\mathbf{y}$, and initial kernel parameters (η, Σ^{-1}):
 - The kernel matrix $\mathbf{K}_h$ is computed using $h_{candidate}$.
 - Projection vectors $(\mathbf{Q}_{C_0}(h))_i = \sum_{j \text{ s.t. } y_j = C_0} (\mathbf{K}_h)_{i,j}$ (and similarly for class C_1 , using the first two unique classes) are computed.
 - The objective score $S(h)$ is calculated using the selected objective metric (spatial or axis) on these projection vectors and labels.
 - The function returns $-S(h)$ for minimization purposes. It returns ∞ if $S(h)$ is NaN or if fewer than two classes are present.

3) **Model Training Procedure:** Given training data $\mathbf{X}$ and labels $\mathbf{y}$:

- $\mathbf{X}_{train} = \mathbf{X}$ is stored, and unique classes $\mathcal{C}$ are identified.
- Initial kernel parameters (η, Σ^{-1}) are estimated using the chosen kernel fit type.
- h is optimized to obtain $h_{opt} = \arg \max_{h \in [h_{min}, h_{max}]} S(h)$ through the minimization of $-S(h)$.
- The final training kernel $\mathbf{K}_{train_opt}$ is computed using h_{opt} , η , and Σ^{-1} with $\mathbf{X}_{train}$.
- A standard SVM (e.g., 'sklearn.svm.SVC') with 'kernel='precomputed' is trained using $\mathbf{K}_{train_opt}$ and $\mathbf{y}$.

4) **Prediction Procedure:** Given new samples $\mathbf{X}_{new}$:

- The test kernel matrix $\mathbf{K}_{test}$ is computed between $\mathbf{X}_{new}$ and $\mathbf{X}_{train}$ using the stored optimal parameters $(h_{opt}, \eta, \Sigma^{-1})$.
- Predicted labels $\hat{\mathbf{y}}_{new}$ are obtained using the trained SVM on $\mathbf{K}_{test}$.

E. Performance Evaluation Framework

The comparison of model performance adheres to a standardized framework.

- **Setup:** Stratified K-fold cross-validation is employed with $K_{cv} = 10$ folds. A significance level $\alpha_{equiv} = 0.05$ is used for an equivalence heuristic. Three model configurations are compared: one using the custom classifier with the 'spatial' objective and 'scale' initial parameter estimation type, another using the custom classifier with the 'axis' objective and 'scale' initial parameter estimation type, and a reference model using a standard SVM, all preceded by the same preprocessing sequence.
- **Evaluation:** For each fold k , models are trained on the training set $\mathcal{T}_k$ and evaluated on the validation set $\mathcal{V}_k$ to obtain accuracy $A_k(M)$.
- **Metrics:** Mean accuracy $\bar{A}(M)$ and standard deviation of accuracy $s_A(M)$ are reported. The total computation time for the cross-validation process is also recorded.
- **Statistical Comparison:** A paired samples t-test is applied to the sets of cross-validation accuracy scores $A_k(M_1)$ and $A_k(M_2)$ to test the null hypothesis $H_0 : \mu_d = 0$ (i.e., the true mean of differences is zero). Models are deemed "statistically equivalent" if the p-value is greater than α_{equiv} .

III. EXPERIMENTAL RESULTS AND DISCUSSION

The performance of the custom kernel support vector classifier, configured with either the spatial or axis objective metric (denoted $M_{spatial}$ and M_{axis} respectively), was compared against a reference SVM model (M_{ref}). All models utilized the same sequential preprocessing. Experiments were conducted with both float32 and float64 numerical precision. The discussion focuses on results from 19 datasets, as the "Adult" dataset led to errors for the custom models.

Complete tabular results are presented in Table I for float32 and Table II for float64 precision.

Accuracy Comparison Summary: With float64 precision, $M_{spatial}$ was statistically significantly worse than M_{ref} on 3 datasets and equivalent on 16. M_{axis} was significantly worse on 2 datasets and equivalent on 17. Neither custom model significantly outperformed the reference. When comparing $M_{spatial}$ to M_{axis} , $M_{spatial}$ was better on 3 datasets, M_{axis} better on 1, and they were equivalent on 15.

With float32 precision, the custom models showed more instances of underperformance: $M_{spatial}$ was worse than M_{ref} on 7 datasets (equivalent on 12), and M_{axis} was worse than M_{ref} on 8 datasets (equivalent on 11). $M_{spatial}$ was better than M_{axis} on 3 datasets, and they were equivalent on 16.

The use of float64 precision appeared to improve the stability of the custom models, leading to fewer instances of statistically significant underperformance compared to the reference model. For example, on datasets like German Credit, Spambase, Digits (0 vs 1), and Digits (5 vs Rest), custom models were notably worse with float32 but showed equivalent performance to the reference model with float64. Despite this, a consistent accuracy advantage over the standard SVM was not demonstrated.

Computation Time: Irrespective of precision, the custom models ($M_{spatial}$ and M_{axis}) consistently required significantly more computation time than M_{ref} . This substantial increase is attributable to the internal hyperparameter optimization loop within the custom kernel classifier. For example, on the Mushroom dataset, $M_{spatial}$ (float64) took 316.61s and M_{axis} (float64) took 756.65s, while M_{ref} took only 4.10s. The float32 versions were generally marginally faster than their float64 counterparts but still orders of magnitude slower than the reference.

Dataset-Specific Observations: The "Adult" dataset resulted in errors for both custom models across both precisions, indicating a persistent issue unrelated to the tested floating-point precision for this specific dataset. On the "Sylvine" dataset, with float64 precision, $M_{spatial}$ performed poorly, while M_{axis} achieved high accuracy, a notable divergence between the two custom approaches. For several datasets, such as "Iris (Setosa vs Versicolor)" and "Banknote Authentication" (for $M_{spatial}$ vs M_{ref}), all models achieved perfect or near-perfect scores and were statistically equivalent.

The overall findings suggest that while the custom kernel optimization offers a structured approach to hyperparameter tuning, its current implementations did not yield a consistent accuracy improvement over a standard SVM and imposed a considerable computational burden. The choice of numerical precision can affect the outcome, with float64 potentially offering more stable optimization for the custom methods.

IV. CONCLUSION

This paper introduced a custom kernel support vector classifier featuring a parameterized kernel, where the kernel's bandwidth hyperparameter undergoes optimization based on

TABLE I
EXPERIMENTAL RESULTS WITH `float32` PRECISION

Dataset	Mean Accuracy $\pm$ Std. Dev.			Time (s)			$M_{spatial}$ vs M_{ref}		M_{axis} vs M_{ref}		$M_{spatial}$ vs M_{axis}	
	$M_{spatial}$	M_{axis}	M_{ref}	$M_{spatial}$	M_{axis}	M_{ref}	p-value	Equiv.	p-value	Equiv.	p-value	Equiv.
Sylvine	0.5015 $\pm$ 0.0082	0.5015 $\pm$ 0.0082	1.0000 $\pm$ 0.0000	6.42	5.45	0.28	0.001	False	0.001	False	1.000	True
Breast Cancer	0.9297 $\pm$ 0.0368	0.9262 $\pm$ 0.0349	0.9332 $\pm$ 0.0375	0.55	0.60	0.04	0.168	True	0.443	True	0.705	True
Banknote Auth.	1.0000 $\pm$ 0.0000	0.9788 $\pm$ 0.0128	1.0000 $\pm$ 0.0000	3.26	4.15	0.05	1.000	True	0.001	False	0.001	False
QSAR Biodeg.	0.8759 $\pm$ 0.0250	0.8408 $\pm$ 0.0913	0.8825 $\pm$ 0.0304	3.17	3.85	0.11	0.111	True	0.165	True	0.224	True
German Credit	0.7000 $\pm$ 0.0000	0.7000 $\pm$ 0.0000	0.7680 $\pm$ 0.0293	4.93	4.72	0.17	0.001	False	0.001	False	1.000	True
Sonar	0.6631 $\pm$ 0.1068	0.7019 $\pm$ 0.0764	0.7064 $\pm$ 0.0939	0.14	0.17	0.03	0.004	False	0.811	True	0.180	True
Spambase	0.6060 $\pm$ 0.0009	0.6060 $\pm$ 0.0009	0.9335 $\pm$ 0.0094	114.50	103.98	1.75	0.001	False	0.001	False	1.000	True
Diabetes	0.7670 $\pm$ 0.0414	0.7813 $\pm$ 0.0381	0.7657 $\pm$ 0.0396	1.00	1.10	0.07	0.723	True	0.212	True	0.223	True
Titanic	0.9458 $\pm$ 0.0134	0.9427 $\pm$ 0.0138	0.9442 $\pm$ 0.0133	3.62	4.14	0.54	0.168	True	0.343	True	0.037	False
Ionosphere	0.9402 $\pm$ 0.0413	0.9487 $\pm$ 0.0439	0.9458 $\pm$ 0.0433	0.49	0.42	0.04	0.168	True	0.577	True	0.081	True
Blood Transfusion	0.7768 $\pm$ 0.0167	0.7647 $\pm$ 0.0089	0.7848 $\pm$ 0.0193	0.86	0.90	0.06	0.081	True	0.009	False	0.041	False
KC1	0.8549 $\pm$ 0.0074	0.8511 $\pm$ 0.0064	0.8535 $\pm$ 0.0078	8.56	10.19	0.23	0.393	True	0.274	True	0.070	True
WPBC	0.7624 $\pm$ 0.0522	0.7483 $\pm$ 0.0401	0.7520 $\pm$ 0.0603	0.25	0.29	0.04	0.344	True	0.831	True	0.302	True
Iris (Setosa vs Rest)	0.9933 $\pm$ 0.0200	1.0000 $\pm$ 0.0000	0.9933 $\pm$ 0.0200	0.08	0.10	0.03	1.000	True	0.343	True	0.343	True
Adult	Error	Error	0.8515 $\pm$ 0.0034	15.34	17.63	296.53	N/A	N/A	N/A	N/A	N/A	N/A
Digits (0 vs 1)	0.5056 $\pm$ 0.0111	0.5056 $\pm$ 0.0111	0.9972 $\pm$ 0.0083	0.73	0.66	0.05	0.001	False	0.001	False	1.000	True
Vote	0.9564 $\pm$ 0.0238	0.9610 $\pm$ 0.0250	0.9588 $\pm$ 0.0222	0.36	0.40	0.04	0.343	True	0.591	True	0.341	True
Mushroom	0.5180 $\pm$ 0.0005	0.5180 $\pm$ 0.0005	1.0000 $\pm$ 0.0000	402.77	394.42	3.75	0.001	False	0.001	False	1.000	True
Digits (5 vs Rest)	0.8987 $\pm$ 0.0022	0.8987 $\pm$ 0.0022	0.9967 $\pm$ 0.0027	15.40	14.88	0.16	0.001	False	0.001	False	1.000	True
Iris (Setosa vs Versicolor)	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	0.09	0.11	0.03	1.000	True	1.000	True	1.000	True

Dataset name abbreviations: Banknote Auth. (Banknote Authentication), QSAR Biodeg. (QSAR Biodegradation), Blood Transfusion (Blood Transfusion Service Center).
"Error" indicates error or no scores reported for accuracy. "N/A" for p-values and equivalence in such cases.

TABLE II
EXPERIMENTAL RESULTS WITH `float64` PRECISION

Dataset	Mean Accuracy $\pm$ Std. Dev.			Time (s)			$M_{spatial}$ vs M_{ref}		M_{axis} vs M_{ref}		$M_{spatial}$ vs M_{axis}	
	$M_{spatial}$	M_{axis}	M_{ref}	$M_{spatial}$	M_{axis}	M_{ref}	p-value	Equiv.	p-value	Equiv.	p-value	Equiv.
Sylvine	0.5015 $\pm$ 0.0082	0.9531 $\pm$ 0.1029	1.0000 $\pm$ 0.0000	7.41	5.78	0.29	0.001	False	0.205	True	0.001	False
Breast Cancer	0.9297 $\pm$ 0.0368	0.9262 $\pm$ 0.0349	0.9332 $\pm$ 0.0375	0.66	1.34	0.04	0.168	True	0.443	True	0.705	True
Banknote Auth.	1.0000 $\pm$ 0.0000	0.9788 $\pm$ 0.0128	1.0000 $\pm$ 0.0000	4.04	8.54	0.05	1.000	True	0.001	False	0.001	False
QSAR Biodeg.	0.8759 $\pm$ 0.0250	0.8825 $\pm$ 0.0286	0.8825 $\pm$ 0.0304	2.47	4.98	0.11	0.111	True	1.000	True	0.088	True
German Credit	0.7660 $\pm$ 0.0294	0.7570 $\pm$ 0.0316	0.7680 $\pm$ 0.0293	2.62	5.27	0.18	0.662	True	0.312	True	0.343	True
Sonar	0.6631 $\pm$ 0.1068	0.7019 $\pm$ 0.0764	0.7064 $\pm$ 0.0939	0.13	0.20	0.04	0.004	False	0.811	True	0.180	True
Spambase	0.9335 $\pm$ 0.0098	0.9322 $\pm$ 0.0072	0.9337 $\pm$ 0.0094	89.90	172.33	1.83	0.841	True	0.353	True	0.508	True
Diabetes	0.7670 $\pm$ 0.0414	0.7813 $\pm$ 0.0381	0.7657 $\pm$ 0.0396	1.17	1.22	0.08	0.723	True	0.212	True	0.223	True
Titanic	0.9458 $\pm$ 0.0134	0.9427 $\pm$ 0.0138	0.9442 $\pm$ 0.0133	3.98	7.48	0.51	0.168	True	0.343	True	0.037	False
Ionosphere	0.9402 $\pm$ 0.0413	0.9487 $\pm$ 0.0439	0.9458 $\pm$ 0.0433	0.63	0.54	0.05	0.168	True	0.577	True	0.081	True
Blood Transfusion	0.7768 $\pm$ 0.0167	0.7647 $\pm$ 0.0089	0.7848 $\pm$ 0.0193	1.10	1.31	0.06	0.081	True	0.009	False	0.041	False
KC1	0.8549 $\pm$ 0.0074	0.8511 $\pm$ 0.0064	0.8535 $\pm$ 0.0078	10.34	20.65	0.24	0.393	True	0.274	True	0.070	True
WPBC	0.7624 $\pm$ 0.0522	0.7483 $\pm$ 0.0401	0.7520 $\pm$ 0.0603	0.21	0.38	0.04	0.344	True	0.831	True	0.302	True
Iris (Setosa vs Rest)	0.9933 $\pm$ 0.0200	1.0000 $\pm$ 0.0000	0.9933 $\pm$ 0.0200	0.08	0.13	0.03	1.000	True	0.343	True	0.343	True
Adult	Error	Error	0.8515 $\pm$ 0.0034	16.93	17.94	304.41	N/A	N/A	N/A	N/A	N/A	N/A
Digits (0 vs 1)	1.0000 $\pm$ 0.0000	0.9944 $\pm$ 0.0111	0.9972 $\pm$ 0.0083	0.39	0.68	0.05	0.343	True	0.591	True	0.168	True
Vote	0.9564 $\pm$ 0.0238	0.9610 $\pm$ 0.0250	0.9588 $\pm$ 0.0222	0.38	0.70	0.04	0.343	True	0.591	True	0.341	True
Mushroom	0.9990 $\pm$ 0.0011	0.9998 $\pm$ 0.0007	1.0000 $\pm$ 0.0000	316.61	756.65	4.10	0.022	False	0.343	True	0.051	True
Digits (5 vs Rest)	0.9955 $\pm$ 0.0033	0.9950 $\pm$ 0.0063	0.9967 $\pm$ 0.0027	7.32	17.18	0.18	0.168	True	0.395	True	0.782	True
Iris (Setosa vs Versicolor)	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	0.15	0.13	0.04	1.000	True	1.000	True	1.000	True

Dataset name abbreviations: Banknote Auth. (Banknote Authentication), QSAR Biodeg. (QSAR Biodegradation), Blood Transfusion (Blood Transfusion Service Center).
"Error" indicates error or no scores reported for accuracy. "N/A" for p-values and equivalence in such cases.

either a spatial or an axis-derived objective function. This classifier was evaluated within a pipeline including standardized preprocessing steps, against a conventional SVM.

Experimental results across a range of datasets and with both `float32` and `float64` precision indicate that the custom kernel approaches, while often achieving statistical equivalence in accuracy to the standard SVM (particularly with `float64` precision), did not consistently surpass it. In some scenarios, especially with `float32` precision, the custom models exhibited significantly poorer performance. A major characteristic of the custom models is their substantially increased computational demand due to the iterative nature of the internal hyperparameter optimization. The use of `float64` precision appeared to mitigate some of the more severe performance degradations observed with `float32`, suggesting a sensitivity of the optimization process to numerical precision.

Future research could focus on the development of alternative objective functions that might correlate more directly with SVM classification performance or exhibit greater robustness across diverse dataset characteristics. Exploring more computationally efficient optimization algorithms for the kernel hyperparameter is crucial for enhancing the practical applicability of such custom kernel methods. Further investigation into the specific dataset characteristics that lead to poor performance or errors in the custom models could also yield valuable insights for refinement.

REFERENCES

- [1] M. V.F. Menezes, L. C.B. Torres, and A. P. Braga, "Width optimization of RBF kernels for binary classification of support vector machines: A density estimation-based approach," *Pattern Recognition Letters*, vol. 128, pp. 1–7, 2019.

- [2] Dua, D. and Graff, C. (2019). UCI Machine Learning Repository [<http://archive.ics.uci.edu/ml>]. Irvine, CA: University of California, School of Information and Computer Science.
- [3] Mansouri, K., Ringsted, T., Ballabio, D., Todeschini, R., Consonni, V. (2013). Quantitative Structure - Activity Relationship models for ready biodegradability of chemicals. *Journal of Chemical Information and Modeling*, 53, 867-878.
- [4] NASA Metrics Data Program (MDP) Datasets. (Various years). Software metrics repositories such as PROMISE provide access to these datasets, e.g., <http://promise.site.uottawa.ca/SERepository/>
- [5] Vanschoren, J., van Rijn, J. N., Bischl, B., & Torgo, L. (2013). OpenML: Networked science in machine learning. *SIGKDD Explorations*, 15(2), 49-60.